Algoritmos e Lógica de Programação: A Bíblia do Kotlin

Versão: 2.0 (Edição Ampliada e Modernizada)
Base Original: Prof. Rogério Ferreira Sgoti (FATEC Botucatu)
Adaptação e Expansão: Engenharia de Software & Design Instrucional

---

SUMÁRIO ANALÍTICO

PARTE 1 — FUNDAMENTAÇÃO TEÓRICA E KOTLIN MODERN

1. Introdução ao Estudo dos Algoritmos
2. Representação de Algoritmos (Fluxogramas a Código)
3. O Ecossistema Kotlin e a JVM (Java Virtual Machine)
4. Estruturas de Dados, Variáveis e Constantes
5. Operadores: Aritméticos, Relacionais e Lógicos
6. Instruções de Entrada e Saída de Dados
7. Estruturas Condicionais (Simples, Compostas, Encadeadas)
8. O Poder do `when` (Múltipla Escolha Modernizada)
9. Estruturas de Repetição (`for` , `while` , `do-while`)
10. Vetores (Arrays) e Coleções Modernas
11. Modularização: O Coração das Funções

PARTE 2 — MASTER LIST: 170+ EXERCÍCIOS PRÁTICOS

- Lista Completa por Tópico

PARTE 1 — TEORIA DETALHADA

1 – INTRODUÇÃO AO ESTUDO DOS ALGORITMOS

1.1 – Definições Fundamentais
Como define Forbellone (2000), um algoritmo é uma "sequência de passos que visa atingir um objetivo bem definido". No contexto do Kotlin, não estamos apenas escrevendo passos, mas construindo soluções seguras e expressivas.

- **Definição Formal:** Um procedimento computacional bem definido que toma valores como entrada e produz valores como saída.

- **A Visão de Dijkstra:** "A arte de programar consiste na arte de organizar e dominar a complexidade". O Kotlin nos ajuda nisso através de sua concisão.

1.2 – Algoritmos Computacionais vs. Não-Computacionais

- **Não-Computacionais:** Processos manuais (ex: trocar um pneu, fazer um café). O foco é a lógica humana.
- **Computacionais:** Processos executados por máquinas. Exigem precisão absoluta na tipagem e na sintaxe.

1.3 – Por que Kotlin?

Diferente do Pascal (usado na apostila original), o Kotlin é:

1. **Interoperável:** Funciona 100% com Java.
2. **Seguro:** Sistema de tipos que evita erros de "Null Pointer".
3. **Expressivo:** Você escreve menos código para fazer o mesmo que o Java ou Pascal.

---

2 – REPRESENTAÇÃO DE ALGORITMOS

Existem três formas principais de representar a lógica antes (ou durante) a codificação:

1. **Linguagem Natural:** Descrição em português. Útil para entender o problema, mas ambígua para a máquina.
2. **Fluxogramas:** Representação gráfica usando símbolos (Losangos para decisão, Retângulos para ação).
3. **Pseudolinguagem (Portugol):** Um meio termo entre o português e o código. No Kotlin, a sintaxe é tão limpa que o próprio código muitas vezes serve como pseudocódigo legível.

---

3 – FUNCIONAMENTO BÁSICO DA JVM

Ao contrário do Pascal que compila para código nativo, o Kotlin compila para **Bytecode**.

- **JVM (Java Virtual Machine):** É a máquina virtual que executa o Bytecode em qualquer sistema (Windows, Linux, Mac).
- **Memória RAM:** Onde as variáveis residem durante a execução.
- **CPU:** Onde os cálculos matemáticos e lógicos são processados.

---

4 – ESTRUTURAS DE DADOS E VARIÁVEIS

4.1 – Declaração: `val` vs `var`

Este é o conceito mais importante do Kotlin Moderno:

- `val` **(Imutável):** Valor constante. Uma vez atribuído, não muda. **Prefira sempre** `val`**.**
- `var` **(Mutável):** Variável comum. Pode ter seu valor alterado.

```
val pi = 3.1415        // Imutável
var contador = 0       // Mutável
contador = contador + 1 // Ok
// pi = 3.15            // Erro de Compilação!
```

4.2 – Tipos de Dados Primitivos (e Objetos)

O Kotlin trata tudo como objeto, o que facilita o uso de funções utilitárias:

- **Inteiros:** `Int` (32 bits), `Long` (64 bits), `Byte`, `Short`.
- **Decimais:** `Double` (padrão), `Float`.
- **Lógicos:** `Boolean` (`true` ou `false`).
- **Texto:** `String` (Textos longos) e `Char` (Único caractere).

4.3 – Null Safety (Segurança de Nulo)

No Kotlin, uma variável não pode receber `null` a menos que você autorize explicitamente usando o ponto de interrogação `?`.

```
var nome: String = "Jose"
// nome = null // Erro!

var apelido: String? = "Zezinho"
apelido = null // Permitido
```

---

5 – OPERADORES E PRIORIDADES

5.1 – Operadores Matemáticos

- `+` : Adição
- `-` : Subtração
- `*` : Multiplicação
- `/` : Divisão
- `%` : Resto da divisão (MOD)

5.2 – Operadores Relacionais (Resultam em Boolean)

- `==` : Igual a
- `!=` : Diferente de

- `>` : Maior que
- `<` : Menor que
- `>=` : Maior ou igual
- `<=` : Menor ou igual

5.3 – Operadores Lógicos

- `&&` (E): Verdadeiro se AMBOS forem verdadeiros.
- `||` (OU): Verdadeiro se PELO MENOS UM for verdadeiro.
- `!` (NÃO): Inverte o valor lógico.

---

6 – INSTRUÇÕES DE ENTRADA E SAÍDA

6.1 – Saída (Print)

Usamos o **String Template** (o símbolo `$`) para inserir variáveis dentro de textos.

```
val idade = 25
println("A idade é $idade anos.") // Exibe: A idade é 25 anos.
println("O dobro da idade é ${idade * 2}.") // Expressões complexas usam
chaves
```

6.2 – Entrada (Read)

```
print("Digite seu nome: ")
val nome = readln() // Lê uma string

print("Digite sua idade: ")
val idade = readln().toInt() // Lê string e converte para Inteiro
```

---

7 – ESTRUTURAS CONDICIONAIS

7.1 – Condicional Simples

```
if (media >= 6.0) {
    println("Parabéns, você passou!")
}
```

7.2 – Condicional Composta

```
if (temperatura > 30) {
    println("Está calor!")
```

```
} else {
    println("Temperatura agradável.")
}
```

7.3 – `if` como Expressão

No Kotlin, o `if` pode retornar um valor, eliminando a necessidade do operador ternário `? :` de outras linguagens.

```
val resultado = if (nota >= 6) "Aprovado" else "Reprovado"
```

---

8 – O PODER DO `WHEN` (Múltipla Escolha)

O `when` substitui o `switch/case` e o `CASO` do Portugol de forma muito mais flexível.

```
when (diaDaSemana) {
    1 -> println("Domingo")
    2 -> println("Segunda")
    in 3..6 -> println("Dias úteis") // Uso de intervalos (Ranges)
    7 -> println("Sábado")
    else -> println("Dia inválido")
}
```

---

9 – ESTRUTURAS DE REPETIÇÃO

9.1 – `while` (Teste no Início)

Executa enquanto a condição for verdadeira. Se a condição for falsa de início, nunca executa.

```
var i = 1
while (i <= 10) {
    println(i)
    i++
}
```

9.2 – `do-while` (Teste no Final)

Garante que o bloco seja executado **pelo menos uma vez**.

```
do {
    println("Executando...")
    val continuar = readln()
} while (continuar == "s")
```

9.3 – `for` e Ranges (Iteração com Controle)

O `for` do Kotlin é baseado em iteradores e intervalos.

```
for (i in 1..10) { ... }       // 1 a 10 (inclusive)
for (i in 1 until 10) { ... }  // 1 a 9
for (i in 10 downTo 1) { ... } // 10 a 1 (decrescente)
for (i in 1..10 step 2) { ... }// 1, 3, 5, 7, 9
```

---

10 – VETORES E COLEÇÕES (Arrays e Lists)

Diferente do Pascal onde o tamanho é fixo e rígido, o Kotlin oferece:

1. **Arrays:** Tamanho fixo. `val numeros = arrayOf(1, 2, 3)`
2. **Lists:** Imutáveis por padrão. `val lista = listOf("A", "B")`
3. **MutableLists:** Podem crescer ou diminuir. `val listaVar = mutableListOf<String>()`

---

11 – MODULARIZAÇÃO (FUNÇÕES)

No Kotlin, não usamos "Procedure" ou "Function" como palavras-chave diferentes. Tudo é `fun`.

Exemplo de Função com Retorno:

```
fun somar(a: Int, b: Int): Int {
    return a + b
}
```

Exemplo de Função de Única Expressão:

```
fun dobrar(n: Int) = n * 2
```

PARTE 2 — MASTER LIST: 170+ EXERCÍCIOS PRÁTICOS

BLOCO 1: FUNDAMENTOS, ENTRADA E SAÍDA (1–20)

Estes exercícios focam em leitura, escrita e cálculos matemáticos básicos.

1. **Olá Mundo:** Exiba a mensagem "Olá, Mundo!" na tela de forma elegante.

2. **Boas-vindas:** Leia o nome de uma pessoa e exiba: "Olá [nome], é um prazer te conhecer!".
3. **Somador Real:** Leia dois números reais e mostre a soma formatada (Ex: "A soma entre 5.5 e 2.1 é 7.6").
4. **Analisador de Tipos:** Leia algo pelo teclado e mostre na tela o seu tipo primitivo e todas as informações possíveis sobre ele (é numérico? é alfabético? está em maiúsculas?).
5. **Antecessor e Sucessor:** Leia um número inteiro e mostre seu antecessor e seu sucessor na mesma linha.
6. **Dobro, Triplo e Raiz:** Leia um número e mostre o seu dobro, triplo e a raiz quadrada (Dica: use `Math.sqrt()`).
7. **Média Aritmética:** Leia duas notas de um aluno, calcule e mostre a sua média.
8. **Conversor de Medidas:** Escreva um programa que leia um valor em metros e o exiba convertido em quilômetros, hectômetros, decâmetros, decímetros, centímetros e milímetros.
9. **Tabuada v1.0:** Leia um número inteiro qualquer e mostre na tela a sua tabuada de 1 a 10.
10. **Conversor de Moedas:** Crie um programa que leia quanto dinheiro uma pessoa tem na carteira (em R$) e mostre quantos dólares ela pode comprar. (Considere US$1.00 = R$5.25 ou valor atual).
11. **Pintando Parede:** Leia a largura e a altura de uma parede em metros, calcule a sua área e a quantidade de tinta necessária para pintá-la, sabendo que cada litro de tinta pinta uma área de 2m².
12. **Calculadora de Descontos:** Faça um algoritmo que leia o preço de um produto e mostre seu novo preço com 5% de desconto.
13. **Reajuste Salarial:** Faça um algoritmo que leia o salário de um funcionário e mostre seu novo salário, com 15% de aumento.
14. **Conversor de Temperaturas:** Escreva um programa que converta uma temperatura digitada em ºC e converta para ºF.
15. **Aluguel de Carros:** Escreva um programa que pergunte a quantidade de Km percorridos por um carro alugado e a quantidade de dias pelos quais ele foi alugado. Calcule o preço a pagar, sabendo que o carro custa R$60 por dia e R$0,15 por Km rodado.
16. **Quebrando um Número:** Crie um programa que leia um número Real qualquer pelo teclado e mostre na tela a sua porção Inteira.
17. **Catetos e Hipotenusa:** Faça um programa que leia o comprimento do cateto oposto e do cateto adjacente de um triângulo retângulo. Calcule e mostre o comprimento da hipotenusa.
18. **Seno, Cosseno e Tangente:** Faça um programa que leia um ângulo qualquer e mostre na tela o valor do seno, cosseno e tangente desse ângulo.
19. **Sorteando um Item:** Um professor quer sortear um dos seus quatro alunos para apagar o quadro. Faça um programa que ajude ele, lendo o nome dos alunos e escrevendo o

nome do escolhido.

20. **Sorteando uma Ordem:** O mesmo professor do desafio 19 quer sortear a ordem de apresentação de trabalhos dos alunos. Faça um programa que leia o nome dos quatro alunos e mostre a ordem sorteada.

BLOCO 2: MANIPULAÇÃO DE TEXTO (21–25)

O Kotlin possui funções poderosas para Strings.

21. **Analisador de Textos:** Crie um programa que leia o nome completo de uma pessoa e mostre:
    - O nome com todas as letras maiúsculas e minúsculas.
    - Quantas letras ao todo (sem considerar espaços).
    - Quantas letras tem o primeiro nome.
22. **Separando Dígitos de um Número:** Faça um programa que leia um número de 0 a 9999 e mostre na tela cada um dos dígitos separados. (Ex: Unidade: 4, Dezena: 3, Centena: 8, Milhar: 1).
23. **Verificando as Primeiras Letras:** Crie um programa que leia o nome de uma cidade e diga se ela começa ou não com o nome "SANTO".
24. **Procurando uma String:** Crie um programa que leia o nome de uma pessoa e diga se ela tem "SILVA" no nome.
25. **Primeira e Última Ocorrência:** Faça um programa que leia uma frase pelo teclado e mostre quantas vezes aparece a letra "A", em que posição ela aparece a primeira vez e em que posição ela aparece a última vez.

BLOCO 3: ESTRUTURAS CONDICIONAIS (26–45)

Decisões simples, compostas e encadeadas.

26. **Radar Eletrônico:** Escreva um programa que leia a velocidade de um carro. Se ele ultrapassar 80Km/h, mostre uma mensagem dizendo que ele foi multado. A multa vai custar R$7,00 por cada Km acima do limite.
27. **Par ou Ímpar?:** Crie um programa que leia um número inteiro e mostre na tela se ele é PAR ou ÍMPAR.
28. **Custo da Passagem:** Desenvolva um programa que pergunte a distância de uma viagem em Km. Calcule o preço da passagem, cobrando R$0,50 por Km para viagens de até 200Km e R$0,45 parta viagens mais longas.
29. **Ano Bissexto:** Faça um programa que leia um ano qualquer e mostre se ele é bissexto.
30. **Maior e Menor Valores:** Faça um programa que leia três números e mostre qual é o maior e qual é o menor.

31. **Aumentos Múltiplos:** Escreva um programa que pergunte o salário de um funcionário e calcule o valor do seu aumento. Para salários superiores a R$1250,00, calcule um aumento de 10%. Para os inferiores ou iguais, o aumento é de 15%.
32. **Analisando Triângulo v1.0:** Desenvolva um programa que leia o comprimento de três retas e diga ao usuário se elas podem ou não formar um triângulo.
33. **Aprovando Empréstimo:** Escreva um programa para aprovar o empréstimo bancário para a compra de uma casa. Pergunte o valor da casa, o salário do comprador e em quantos anos ele vai pagar. A prestação mensal não pode exceder 30% do salário ou então o empréstimo será negado.
34. **Conversor de Bases Numéricas:** Escreva um programa em Kotlin que leia um número inteiro qualquer e peça para o usuário escolher qual será a base de conversão: 1 para binário, 2 para octal e 3 para hexadecimal.
35. **Comparando Números:** Escreva um programa que leia dois números inteiros e compare-os. mostrando na tela uma mensagem: "O primeiro valor é maior", "O segundo valor é maior" ou "Não existe valor maior, os dois são iguais".
36. **Alistamento Militar:** Faça um programa que leia o ano de nascimento de um jovem e informe, de acordo com a sua idade, se ele ainda vai se alistar ao serviço militar, se é a hora exata de se alistar ou se já passou do tempo do alistamento. Seu programa também deverá mostrar o tempo que falta ou que passou do prazo.
37. **Aquele Clássico da Média:** Crie um programa que leia duas notas de um aluno e calcule sua média, mostrando uma mensagem no final, de acordo com a média atingida: Média abaixo de 5.0: REPROVADO; Média entre 5.0 e 6.9: RECUPERAÇÃO; Média 7.0 ou superior: APROVADO.
38. **Classificando Atletas:** A Confederação Nacional de Natação precisa de um programa que leia o ano de nascimento de um atleta e mostre sua categoria, de acordo com a idade: Até 9 anos: MIRIM; Até 14 anos: INFANTIL; Até 19 anos: JÚNIOR; Até 25 anos: SÊNIOR; Acima de 25 anos: MASTER.
39. **Analisando Triângulos v2.0:** Refaça o DESAFIO 32 dos triângulos, acrescentando o recurso de mostrar que tipo de triângulo será formado: EQUILÁTERO: todos os lados iguais; ISÓSCELES: dois lados iguais, um diferente; ESCALENO: todos os lados diferentes.
40. **Índice de Massa Corporal (IMC):** Desenvolva uma lógica que leia o peso e a altura de uma pessoa, calcule seu Índice de Massa Corporal (IMC) e mostre seu status, de acordo com a tabela abaixo: Abaixo de 18.5: Abaixo do Peso; Entre 18.5 e 25: Peso Ideal; 25 até 30: Sobrepeso; 30 até 40: Obesidade; Acima de 40: Obesidade Mórbida.
41. **Gerenciador de Pagamentos:** Elabore um programa que calcule o valor a ser pago por um produto, considerando o seu preço normal e condição de pagamento: à vista dinheiro/cheque: 10% de desconto; à vista no cartão: 5% de desconto; em até 2x no cartão: preço normal; 3x ou mais no cartão: 20% de juros.
42. **GAME: Pedra Papel e Tesoura (Jokenpô):** Crie um programa que faça o computador jogar Jokenpô com você.

43. **Menu de Opções:** Crie um programa que leia dois valores e mostre um menu na tela: [1] somar, [2] multiplicar, [3] maior, [4] novos números, [5] sair do programa. Seu programa deverá realizar a operação solicitada em cada caso.
44. **Cálculo de Fatorial:** Faça um programa que leia um número qualquer e mostre o seu fatorial. Ex: 5! = 5 x 4 x 3 x 2 x 1 = 120.
45. **Progressão Aritmética v2.0:** Refaça o DESAFIO 51 (abaixo), lendo o primeiro termo e a razão de uma PA, mostrando os 10 primeiros termos da progressão usando a estrutura `while`.

BLOCO 4: ESTRUTURAS DE REPETIÇÃO (46–70)

Onde a lógica ganha escala.

46. **Contagem Regressiva:** Faça um programa que mostre na tela uma contagem regressiva para o estouro de fogos de artifício, indo de 10 até 0, com uma pausa de 1 segundo entre eles (Use `Thread.sleep(1000)`).
47. **Contagem de Pares:** Crie um programa que mostre na tela todos os números pares que estão no intervalo entre 1 e 50.
48. **Soma de Ímpares Múltiplos de Três:** Faça um programa que calcule a soma entre todos os números que são ímpares, são múltiplos de três e que se encontram no intervalo de 1 até 500.
49. **Tabuada v2.0 (com `for`):** Refaça o DESAFIO 9, mostrando a tabuada de um número que o usuário escolher, só que agora utilizando um laço `for`.
50. **Soma dos Pares:** Desenvolva um programa que leia seis números inteiros e mostre a soma apenas daqueles que forem pares. Se o valor digitado for ímpar, desconsidere-o.
51. **Progressão Aritmética v1.0:** Desenvolva um programa que leia o primeiro termo e a razão de uma PA. No final, mostre os 10 primeiros termos dessa progressão.
52. **Números Primos:** Faça um programa que leia um número inteiro e diga se ele é ou não um número primo.
53. **Detector de Palíndromo:** Crie um programa que leia uma frase qualquer e diga se ela é um palíndromo, desconsiderando os espaços. (Ex: APOS A SOPA, A SACADA DA CASA, A TORRE DA DERROTA).
54. **Grupo da Maioridade:** Crie um programa que leia o ano de nascimento de sete pessoas. No final, mostre quantas pessoas ainda não atingiram a maioridade e quantas já são maiores. (Considere maioridade = 21 anos).
55. **Maior e Menor da Sequência:** Faça um programa que leia o peso de cinco pessoas. No final, mostre qual foi o maior e o menor peso lidos.
56. **Analisador Completo:** Desenvolva um programa que leia o nome, idade e sexo de 4 pessoas. No final do programa, mostre: a média de idade do grupo, qual é o nome do homem mais velho e quantas mulheres têm menos de 20 anos.

57. **Validação de Dados:** Faça um programa que leia o sexo de uma pessoa, mas só aceite os valores 'M' ou 'F'. Caso esteja errado, peça a digitação novamente até ter um valor correto.
58. **Jogo da Adivinhação v2.0:** Melhore o jogo do DESAFIO 28 onde o computador vai "pensar" em um número entre 0 e 10. Só que agora o jogador vai tentar adivinhar até acertar, mostrando no final quantos palpites foram necessários para vencer.
59. **Cálculo de Fatorial v2.0:** Refaça o exercício 44 usando `for` e `while` para comparar.
60. **Super Progressão Aritmética v3.0:** Melhore o DESAFIO 45, perguntando para o usuário se ele quer mostrar mais alguns termos. O programa encerrará quando ele disser que quer mostrar 0 termos.
61. **Sequência de Fibonacci v1.0:** Escreva um programa que leia um número N inteiro qualquer e mostre na tela os N primeiros elementos de uma Sequência de Fibonacci. Ex: 0 – 1 – 1 – 2 – 3 – 5 – 8.
62. **Tratando Vários Valores v1.0:** Crie um programa que leia vários números inteiros pelo teclado. O programa só vai parar quando o usuário digitar o valor 999, que é a condição de parada. No final, mostre quantos números foram digitados e qual foi a soma entre eles (desconsiderando o flag).
63. **Maior e Menor Valores v2.0:** Crie um programa que leia vários números inteiros pelo teclado. No final da execução, mostre a média entre todos os valores e qual foi o maior e o menor valores lidos. O programa deve perguntar ao usuário se ele quer ou não continuar a digitar valores.
64. **Vários Números com Flag:** Crie um programa que leia números inteiros pelo teclado. O programa só vai parar quando o usuário digitar o valor 999. No final, mostre quantos números foram digitados e qual foi a soma entre eles (usando `break`).
65. **Tabuada v3.0 (Infinita):** Faça um programa que mostre a tabuada de vários números, um de cada vez, para cada valor digitado pelo usuário. O programa será interrompido quando o número solicitado for negativo.
66. **Jogo do Par ou Ímpar:** Faça um programa que jogue par ou ímpar com o computador. O jogo só será interrompido quando o jogador perder, mostrando o total de vitórias consecutivas que ele conquistou no final do jogo.
67. **Análise de Dados do Grupo:** Crie um programa que leia a idade e o sexo de várias pessoas. A cada pessoa cadastrada, o programa deverá perguntar se o usuário quer ou não continuar. No final, mostre: A) quantas pessoas tem mais de 18 anos. B) quantos homens foram cadastrados. C) quantas mulheres tem menos de 20 anos.
68. **Estatísticas em Produtos:** Crie um programa que leia o nome e o preço de vários produtos. O programa deverá perguntar se o usuário vai continuar ou não. No final mostre: A) qual é o total gasto na compra. B) quantos produtos custam mais de R$1000. C) qual é o nome do produto mais barato.
69. **Simulador de Caixa Eletrônico:** Crie um programa que simule o funcionamento de um caixa eletrônico. No início, pergunte ao usuário qual será o valor a ser sacado (número inteiro) e o programa vai informar quantas cédulas de cada valor serão entregues. OBS: considere que o caixa possui cédulas de R$50, R$20, R$10 e R$1.

70. **Número por Extenso:** Crie um programa que tenha uma dupla totalmente preenchida com uma contagem por extenso, de zero até vinte. Seu programa deverá ler um número pelo teclado (entre 0 e 20) e mostrá-lo por extenso.

BLOCO 5: VETORES, MATRIZES E COLEÇÕES (71–110)

Trabalhando com múltiplos dados.

71. **Tuplas com Times de Futebol:** Crie uma tupla preenchida com os 20 primeiros colocados da Tabela do Campeonato Brasileiro de Futebol, na ordem de colocação. Depois mostre: a) Os 5 primeiros. b) Os últimos 4 colocados. c) Times em ordem alfabética. d) Em que posição está o time da Chapecoense (ou outro time).
72. **Maior e Menor valores em Tupla:** Crie um programa que vai gerar cinco números aleatórios e colocar em uma tupla. Depois disso, mostre a listagem de números gerados e também indique o menor e o maior valor que estão na tupla.
73. **Análise de dados em uma Tupla:** Desenvolva um programa que leia quatro valores pelo teclado e guarde-os em uma tupla. No final, mostre: A) Quantas vezes apareceu o valor 9. B) Em que posição foi digitado o primeiro valor 3. C) Quais foram os números pares.
74. **Lista de Preços com Tupla:** Crie um programa que tenha uma tupla única com nomes de produtos e seus respectivos preços, na sequência. No final, mostre uma listagem de preços, organizando os dados em forma tabular.
75. **Contando Vogais em Tupla:** Crie um programa que tenha uma tupla com várias palavras (não usar acentos). Depois disso, você deve mostrar, para cada palavra, quais são as suas vogais.
76. **Maior e Menor Valores na Lista:** Faça um programa que leia 5 valores numéricos e guarde-os em uma lista. No final, mostre qual foi o maior e o menor valor digitado e as suas respectivas posições na lista.
77. **Valores Únicos em uma Lista:** Crie um programa onde o usuário possa digitar vários valores numéricos e cadastre-os em uma lista. Caso o número já exista lá dentro, ele não será adicionado. No final, serão exibidos todos os valores únicos digitados, em ordem crescente.
78. **Lista Ordenada sem Repetições:** Crie um programa onde o usuário possa digitar cinco valores numéricos e cadastre-os em uma lista, já na posição correta de inserção (sem usar o `sort()`). No final, mostre a lista ordenada na tela.
79. **Extraindo Dados de uma Lista:** Crie um programa que vai ler vários números e colocar em uma lista. Depois disso, mostre: A) Quantos números foram digitados. B) A lista de valores, ordenada de forma decrescente. C) Se o valor 5 foi digitado e está ou não na lista.
80. **Dividindo Valores em Várias Listas:** Crie um programa que vai ler vários números e colocar em uma lista. Depois disso, crie duas listas extras que vão conter apenas os

valores pares e os valores ímpares digitados, respectivamente. Ao final, mostre o conteúdo das três listas geradas.

81. **Validando Expressões Matemáticas:** Crie um programa onde o usuário digite uma expressão qualquer que use parênteses. Seu aplicativo deverá analisar se a expressão passada está com os parênteses abertos e fechados na ordem correta.
82. **Listas com Composição (Peso):** Faça um programa que leia nome e peso de várias pessoas, guardando tudo em uma lista. No final, mostre: A) Quantas pessoas foram cadastradas. B) Uma listagem com as pessoas mais pesadas. C) Uma listagem com as pessoas mais leves.
83. **Listas com Pares e Ímpares:** Crie um programa onde o usuário possa digitar sete valores numéricos e cadastre-os em uma lista única que mantenha separados os valores pares e ímpares. No final, mostre os valores pares e ímpares em ordem crescente.
84. **Matriz v1.0:** Crie um programa que declare uma matriz de dimensão 3x3 e preencha com valores lidos pelo teclado. No final, mostre a matriz na tela, com a formatação correta.
85. **Mais Sobre Matriz:** Aprimore o desafio anterior, mostrando no final: A) A soma de todos os valores pares digitados. B) A soma dos valores da terceira coluna. C) O maior valor da segunda linha.
86. **Palpites para a Mega Sena:** Faça um programa que ajude um jogador da MEGA SENA a criar palpites.O programa vai perguntar quantos jogos serão gerados e vai sortear 6 números entre 1 e 60 para cada jogo, cadastrando tudo em uma lista composta.
87. **Boletim com Listas Compostas:** Crie um programa que leia nome e duas notas de vários alunos e guarde tudo em uma lista composta. No final, mostre um boletim contendo a média de cada um e permita que o usuário possa mostrar as notas de cada aluno individualmente.
88. **Dicionário em Kotlin (Maps):** Faça um programa que leia nome e média de um aluno, guardando também a situação em um dicionário (Map). No final, mostre o conteúdo da estrutura na tela.
89. **Jogo de Dados:** Crie um programa onde 4 jogadores joguem um dado e tenham resultados aleatórios. Guarde esses resultados em um Map em ordem. No final, coloque esse dicionário em ordem, sabendo que o vencedor tirou o maior número no dado.
90. **Cadastro de Trabalhador:** Crie um programa que leia nome, ano de nascimento e carteira de trabalho e cadastre-os (com idade) em um Map. Se por acaso a CTPS for diferente de ZERO, o dicionário receberá também o ano de contratação e o salário. Calcule e acrescente, além da idade, com quantos anos a pessoa vai se aposentar.
91. **Cadastro de Jogador de Futebol:** Crie um programa que gerencie o aproveitamento de um jogador de futebol. O programa vai ler o nome do jogador e quantas partidas ele jogou. Depois vai ler a quantidade de gols feitos em cada partida. No final, tudo isso será guardado em um Map, incluindo o total de gols feitos durante o campeonato.
92. **Unindo Dicionários e Listas:** Crie um programa que leia nome, sexo e idade de várias pessoas, guardando os dados de cada pessoa em um Map e todos os dicionários em

uma Lista. No final, mostre: A) Quantas pessoas foram cadastradas B) A média de idade C) Uma lista com as mulheres D) Uma lista de pessoas com idade acima da média.

93. **Aprimorando os Dicionários:** Aprimore o desafio 91 para que ele funcione com vários jogadores, incluindo um sistema de visualização de detalhes do aproveitamento de cada jogador.
94. **Função que Calcula Área:** Faça um programa que tenha uma função chamada `area()`, que receba as dimensões de um terreno retangular (largura e comprimento) e mostre a área do terreno.
95. **Um Print Especial:** Faça um programa que tenha uma função chamada `escreva()`, que receba um texto qualquer como parâmetro e mostre uma mensagem com tamanho adaptável. Ex: `escreva("Olá, Mundo!")` Saída:

```
Olá, Mundo!
```

96. **Função de Contador:** Faça um programa que tenha uma função chamada `contador()`, que receba três parâmetros: início, fim e passo. Seu programa tem que realizar três contagens através da função criada: a) de 1 até 10, de 1 em 1 b) de 10 até 0, de 2 em 2 c) uma contagem personalizada.
97. **Função que descobre o maior:** Faça um programa que tenha uma função chamada `maior()`, que receba vários parâmetros com valores inteiros. Seu programa tem que analisar todos os valores e dizer qual deles é o maior.
98. **Funções para sortear e somar:** Faça um programa que tenha uma lista chamada números e duas funções chamadas `sorteia()` e `somaPar()`. A primeira função vai sortear 5 números e vai colocá-los dentro da lista e a segunda função vai mostrar a soma entre todos os valores PARES sorteados pela função anterior.
99. **Funções para votação:** Crie um programa que tenha uma função chamada `voto()` que vai receber como parâmetro o ano de nascimento de uma pessoa, retornando um valor literal indicando se uma pessoa tem voto NEGADO, OPCIONAL e OBRIGATÓRIO nas eleições.
100. **Função para Fatorial:** Crie um programa que tenha uma função `fatorial()` que receba dois parâmetros: o primeiro que indique o número a calcular e outro chamado `show`, que será um valor lógico (opcional) indicando se será mostrado ou não na tela o processo de cálculo do fatorial.
101. **Ficha do Jogador:** Faça um programa que tenha uma função chamada `ficha()`, que receba dois parâmetros opcionais: o nome de um jogador e quantos gols ele marcou. O programa deverá conseguir mostrar a ficha do jogador, mesmo que algum dado não tenha sido informado corretamente.
102. **Função para ler Inteiros:** Crie um programa que tenha a função `leiaInt()`, que vai funcionar semelhante à função `input()` do Kotlin, mas fazendo a validação para aceitar apenas um número inteiro. Ex: `n = leiaInt("Digite um n: ")`
103. **Analisando e gerando Dicionários:** Faça um programa que tenha uma função `notas()` que pode receber várias notas de alunos e vai retornar um Map com as

seguintes informações: — Quantidade de notas — A maior nota — A menor nota — A média da turma — A situação (opcional).

104. **Sistema Interativo de Ajuda:** Faça um mini-sistema que utilize o `Help` do Kotlin (KDoc). O usuário vai digitar o comando e o manual vai aparecer. Quando o usuário digitar a palavra 'FIM', o programa se encerrará. Importante: use cores (ANSI)!
105. **Módulos e Pacotes:** Crie um módulo chamado `moeda.kt` que tenha as funções incorporadas `aumentar()`, `diminuir()`, `dobro()` e `metade()`. Faça também um programa que importe esse módulo e use algumas dessas funções.
106. **Formatando Moedas:** Adapte o código do desafio 105, criando uma função adicional chamada `moeda()` que consiga mostrar os números como um valor monetário formatado.
107. **Reduzindo ainda mais seu programa:** Modifique as funções que foram criadas no desafio 105 para que elas aceitem um parâmetro a mais, informando se o valor retornado por elas vai ser formatado ou não pela função moeda(), desenvolvida no desafio 106.
108. **Resumo do Valor:** Adicione ao módulo `moeda.kt` criado nos desafios anteriores, uma função chamada `resumo()`, que mostre na tela algumas informações geradas pelas funções que já temos no módulo até aqui.
109. **Transformando módulos em pacotes:** Crie um pacote chamado `utilidades` que tenha dois módulos internos chamados `moeda` e `dado`. Transfira todas as funções utilizadas nos desafios 105, 106 e 107 para o primeiro pacote e mantenha tudo funcionando.
110. **Validando entrada de dados monetários:** Dentro do pacote `utilidades` que criamos no desafio 109, temos um módulo chamado `dado`. Crie uma função chamada `leiaDinheiro()` que seja capaz de funcionar como a função `readln()`, mas com uma validação de dados para aceitar apenas valores que sejam monetários.

BLOCO 6: ERROS, ARQUIVOS E COROUTINES (111–115)

Conceitos avançados e persistência.

111. **Funções aprofundadas em Kotlin:** Reescreva a função `leiaInt()` que fizemos no desafio 102, incluindo agora a possibilidade da digitação de um número de tipo inválido sem que o programa trave. Aproveite e crie também uma função `leiaFloat()` com a mesma funcionalidade.
112. **Acessando um Site:** Crie um código em Kotlin que teste se o site Pudim (ou Google) está acessível pelo computador usado.
113. **Sistema de Cadastro v1.0:** Crie um pequeno sistema modularizado que permita cadastrar pessoas pelo seu nome e idade em um arquivo de texto simples.
114. **Sistema de Cadastro v2.0 (Persistência):** Aprimore o sistema anterior para que ele consiga ler os dados do arquivo e mostrá-los em uma tabela organizada.

115. **Introdução às Coroutines:** Crie um programa que execute duas funções de contagem simultaneamente usando `launch` e `delay`.

Este material é de propriedade intelectual adaptada para fins de ensino de lógica de programação. A prática constante é o único caminho para a maestria.